

Práctica 2.1: Introducción a Optimización y explotación Paralelismo de Datos

Índice

Objetivos	1
1 Antes de nada	1
1.1 Compilador Intel® oneAPI DPC++/C++ Compiler	2
Ejercicios	3
Ejercicio 1: Opciones de compilación	3
1.1.1 Tarea	3
Ejercicio 2: Vectorización automática-reportes de compilación	3
1.1.2 Tarea	4
Ejercicio 3: Vectorización automática-Cálculo potencial eléctrico	4
1.1.3 Intel Advisor: Optimización y Análisis de Rendimiento para Aplicaciones de Alto Desempeño	5
1.2 Ejercicio 4: Vectorización guiada	8
1.3 Ejercicio 5: Entrega k-nearest neighbors (KNN)	9
1.3.1 Tarea	9

Objetivos

Esta práctica persigue que los estudiantes se familiaricen con la mejora de rendimiento por la explotación de paralelismo ILP mediante flags de compilación y DLP (SIMD) de forma automática o guiada con pragmas de OpenMP.

1 Antes de nada

- ¿Cómo saber el equipo disponible?
 - Modelo del procesador
 - Número de cores: 8 Performance-cores y 4 Efficient-cores
 - Vectorización: *sse, avx*...
 - Especificaciones a consultar en la página de Intel

```
user@lab:~ $ more /proc/cpuinfo
...
processor : 0
...
model name : 12th Gen Intel(R) Core(TM) i7-12700
...
cpu cores : 16
...
...
```

```
flags      : ... avx ... avx2 ...
....
```

- Uso de P-cores o E-cores con `taskset` para fijar el core donde ejecutar
 - 0-15 usa P-cores
 - 16-19 usa E-cores

```
user@lab:~ $ taskset --cpu-list 0 python3 -c "import numpy as np, time; N=5000; A=np.random.rand(N, N);
Execution time: 2.976552724838257 seconds
user@lab:~ $ taskset --cpu-list 16 python3 -c "import numpy as np, time; N=5000; A=np.random.rand(N, N);
Execution time: 8.270392179489136 seconds
```

1.1 Compilador Intel® oneAPI DPC++/C++ Compiler

Durante la práctica vamos a emplear el compilador Intel® oneAPI DPC++/C++ Compiler, que permite explotar varios niveles de paralelismo que vamos a emplear durante la asignatura. La explotación de paralelismo ILP se puede lograr con flags de optimización tipo `-O[n]`, además para la explotación del paralelismo tipo DLP (SIMD) se logra de forma automática con `-O2` o activando OpenMP con el flag `-fopenmp`, para el paralelismo tipo TLP se activa OpenMP con el flag `-fopenmp`, y por último para el paralelismo tipo GPU `-fiopenmp -fopenmp-targets=spir64`.

El compilador está instalado dentro de la suite de software de Intel-oneAPI, que se puede descargar e instalar en la Toolkit Intel® oneAPI Base Toolkit. Para su activación en el laboratorio se debe de activar las variables de entorno:

```
user@lab:~ $ source /opt/intel/oneapi/setvars.sh

:: initializing oneAPI environment ...
  bash: BASH_VERSION = 5.1.16(1)-release
  args: Using "$@" for setvars.sh arguments:
:: advisor -- latest
:: ccl -- latest
:: compiler -- latest
:: dal -- latest
:: debugger -- latest
:: dev-utilities -- latest
:: dnnl -- latest
:: dpcpp-ct -- latest
:: dpl -- latest
:: ipp -- latest
:: ippcp -- latest
:: mkl -- latest
:: mpi -- latest
:: pti -- latest
:: tbb -- latest
:: umf -- latest
:: vtune -- latest
:: oneAPI environment initialized ::

user@lab:~ $ icpx --version
Intel(R) oneAPI DPC++/C++ Compiler 2025.0.4 (2025.0.4.20241205)
Target: x86_64-unknown-linux-gnu
Thread model: posix
InstalledDir: /opt/intel/oneapi/compiler/2025.0/bin/compiler
Configuration file: /opt/intel/oneapi/compiler/2025.0/bin/compiler/./icpx.cfg
```

Ejercicios

Ejercicio 1: Opciones de compilación

Los “flags de compilación” especifica las optimizaciones en compilación. Por defecto el compilador utilizar la opción -O2, aunque existen otras opciones de compilación que habilitan optimizaciones. Estas opciones se conocen como -O[n]

O0	Deshabilita cualquier optimización
O1	Habilita optimizaciones que incrementan tamaño código pero lo acelera Optimización global: análisis de flujo, análisis de vida de variables Deshabilita inlining y uso de intrínsecas
O2	Activa vectorización Optimización a nivel de bucle Optimización entre funciones
O3	O2+ Opt. más agresiva: fusión de bucles, unroll Adecuada para código basados en bucles y cómputo fp

- Con el fin de poder evaluar las ganancias generadas con las diferentes opciones de optimización tomaremos como ejemplo el código `**foo.c**`.

1.1.1 Tarea

- Evalua los tiempos de ejecución y ganancias con las opciones compilación anteriormente descritas que mejoran la explotación del paralelismo ILP.

Ejercicio 2: Vectorización automática-reportes de compilación

La opción **-O2** del compilador de Intel® oneAPI DPC++/C++ Compiler habilita al autovectorizador: O2 “Vectorization is enabled at O2 and higher levels.”

No obstante existe un flag que activa el reporte del compilación `qopt-report=n` para saber si se ha realizado la vectorización, generando un fichero **.opt rpt** donde se indica las optimizaciones realizadas. La opción de reporte tiene 3 niveles de detalle:

- n=0: Disables generation of an optimization report. This is the default when the option is not specified.
- n=1: Tells the compiler to create a report with minimum details.
- n=2: Tells the compiler to create a report with medium details. This is the default if you do not specify arg.
- n=3: Tells the compiler to create a report with maximum details.

Terminal 1:

```
user@lab:~ $ icx -o fooO2 foo.c -O2 -qopt-report=2
user@lab:~ $ more foo.opt rpt
....
LOOP BEGIN at foo.c (23, 2)
<Multiversiomed vl>
  remark #25228: Loop multiversiomed for Data Dependence
  remark #15300: LOOP WAS VECTORIZED
  remark #15305: vectorization support: vector length 2
LOOP END
```

1.1.2 Tarea

- Evalúa si el compilador es capaz de generar código vectorial para `foo.c` de forma automática.

Ejercicio 3: Vectorización automática-Cálculo potencial eléctrico

Cálculo de potencial eléctrico, suponiendo m partículas identificadas por el índice i . Cada partícula es descrita por las tres coordenadas cartesianas $\vec{r} \equiv (r_{i,x}, r_{i,y}, r_{i,z})$ y la carga q_i . El cálculo del potencial $\phi(\vec{R})$ en múltiples puntos $\vec{R}_j \equiv (R_{j,x}, R_{j,y}, R_{j,z})$, donde j denota una de las n localizaciones, se rige por la ecuación $\phi(R_j) = -\sum_{i=1}^m \frac{q_i}{\sqrt{(r_{i,x}-R_{j,x})^2+(r_{i,y}-R_{j,y})^2+(r_{i,z}-R_{j,z})^2}}$.

Para ello necesitamos calcular el potencial eléctrico $\phi(\vec{R})$, donde j indica 1 de las n posiciones. La figura visualiza el problema.

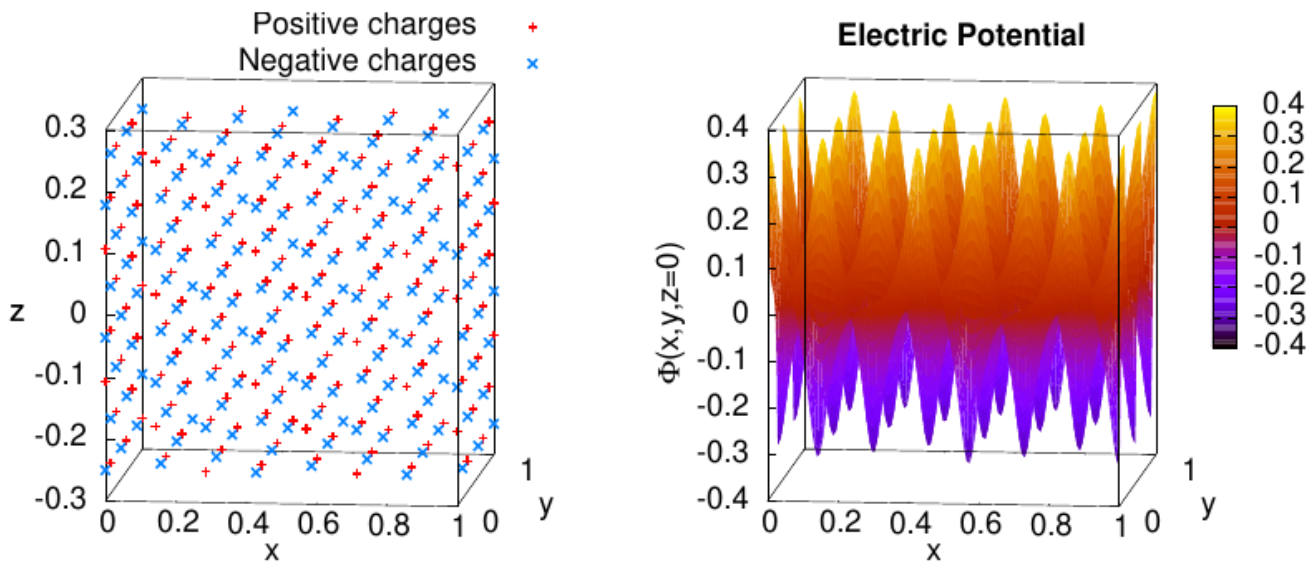


Figure 1: Izda: partículas cargadas. Derecha: Potencial eléctrico ϕ producido por cada partícula

Más información puede encontrarse en el libro donde se ha extraído el ejemplo (pág 290), y el código ha sido extraído del repositorio GitHub.

Para la solución, la codificación emplea una estructura por partícula:

```
struct Charge { // Elegant, but ineffective data layout
    float x, y, z, q; // Coordinates and value of this charge
    int type; // Positive or negative charge
};
```

y el cálculo del potencial eléctrico se puede implementar en la función **CalculateElectricPotential**:

```
void CalculateElectricPotential(
    const int m, // Number of charges
    const Charge* chg, // Charge distribution (array of classes)
    const double Rx, const double Ry, const double Rz, // Observation point
    float & phi // Output: electric potential
) {
    phi=0.0f;
    for (int i=0; i<m; i++) { // This loop will be auto-vectorized
        // Non-unit stride: (&chg[i+1].x - &chg[i].x) == sizeof(Charge)
        const double dx=chg[i].x - Rx;
```

```

const double dy=chg[i].y - Ry;
const double dz=chg[i].z - Rz;
phi -= chg[i].q / sqrt(dx*dx+dy*dy+dz*dz); // Coulomb's law
}
}

```

Existen 4 versiones:

- ver0: versión sin optimizar
 - Conviene prestar atención a las opciones de compilación (añadir arquitectura) y ver ineficiencias (conversiones de datos)
- ver1: optimizado a la arquitectura *target*, pero con ineficiencias de acceso a memoria (stride!=1)
 - Propuesta cambiar AoS por SoA
- ver2: optimización de memoria

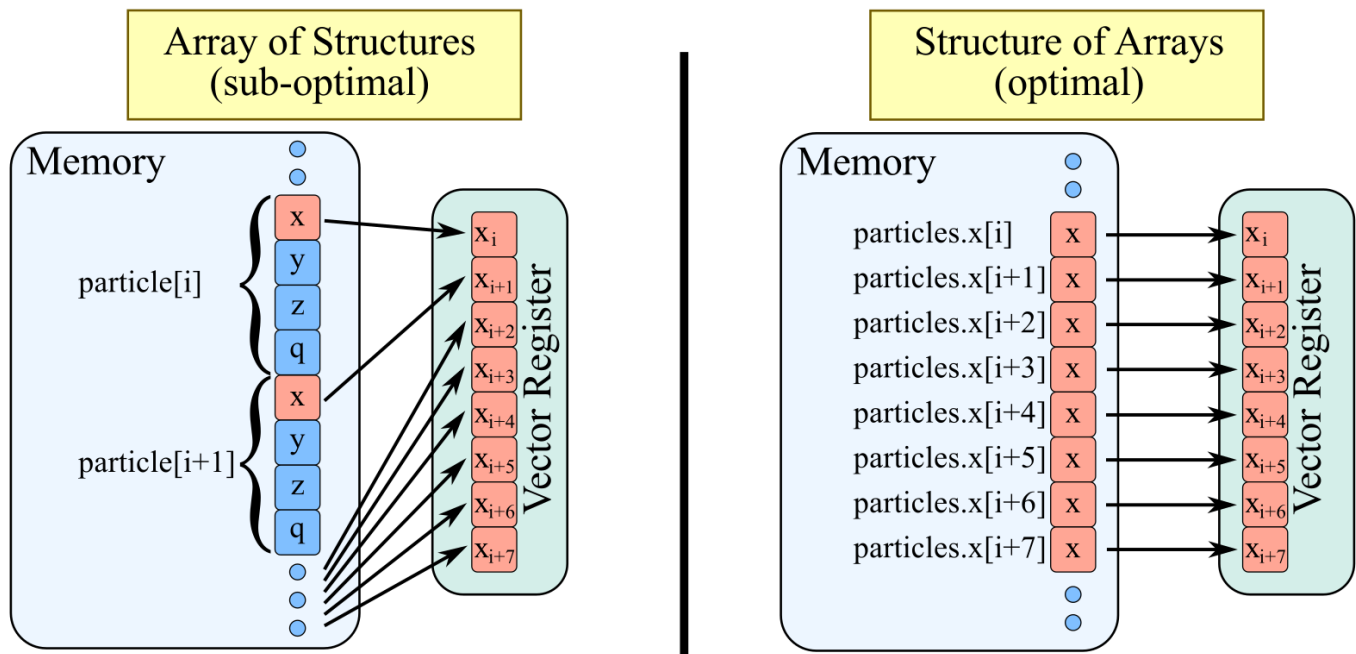


Figure 2: Array of Structure vs Structure of Arrays

- ver3: optimización de memoria a accesos alineados (*compilador no genera accesos alineados*)

Y la compilación de cada versión se puede realizar invocando la herramienta `make`.

1.1.3 Intel Advisor: Optimización y Análisis de Rendimiento para Aplicaciones de Alto Desempeño

Intel Advisor es una potente herramienta de perfilado y optimización diseñada para ayudar a los desarrolladores a mejorar el rendimiento de sus aplicaciones. Proporciona información detallada sobre la ejecución del código, identificando cuellos de botella y oportunidades de optimización, con especial enfoque en la vectorización y paralelización.

Principales Características:

- Análisis de rendimiento avanzado: Permite evaluar la eficiencia de la aplicación mediante la recopilación de métricas clave, ayudando a identificar partes del código que afectan el rendimiento.
- Detección y análisis de bucles vectorizados: Muestra qué bucles están siendo vectorizados correctamente y cuáles pueden beneficiarse de optimización, proporcionando informes detallados con sugerencias específicas.

- Recomendaciones para mejorar la eficiencia: Intel Advisor no solo detecta áreas de mejora, sino que también proporciona recomendaciones basadas en modelos avanzados de análisis para optimizar el uso de recursos del hardware.
- Interfaz intuitiva y generación de reportes: Su interfaz gráfica y opciones de línea de comandos permiten una visualización clara de los datos y la generación de informes detallados para la toma de decisiones.

El uso del Intel Advisor se puede realizar con el comando `advisor-gui`.

1.1.3.1 Ejemplo Intel Advisor

- 1 Botón de control para guardar resultados
- 2 y 3 Filtro de resultados
- 10 Modelo roofline
- 11 Resumen: funciones vectorizadas, bucles vectorizados, función escalar y para bucles escalares
- 13 Panel de recomendaciones (Optimización)
- 14 Razones para la no vectorización

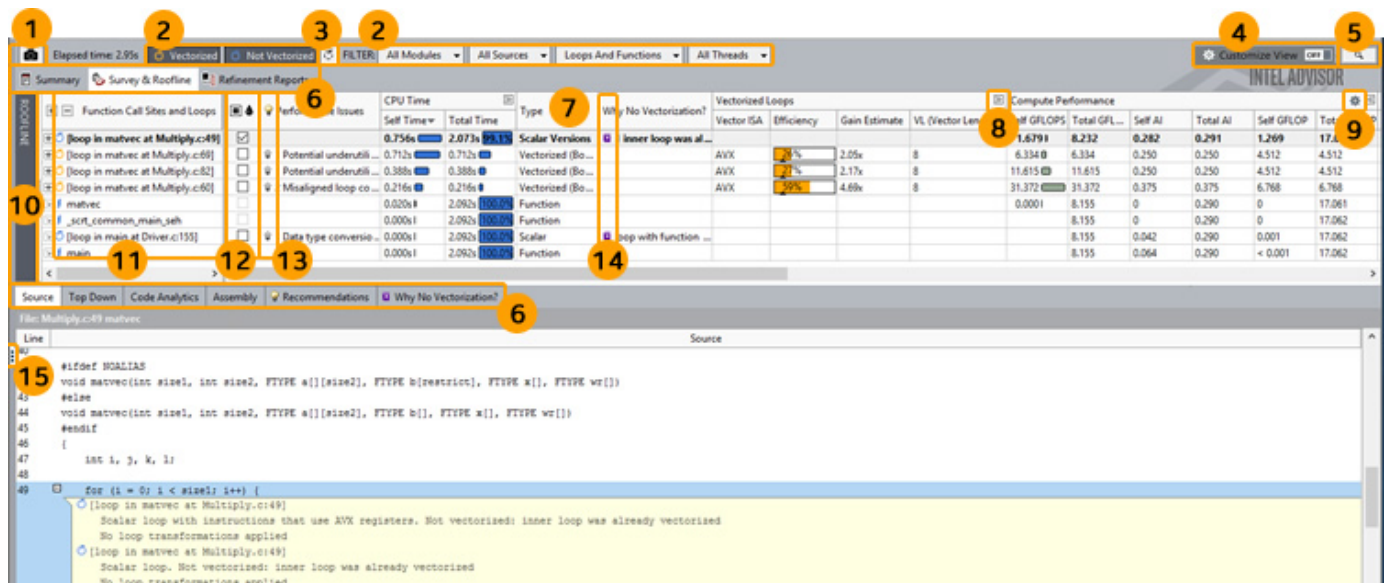


Figure 3: Intel Advisor

1.1.3.2 Panel de Recomendación (recommendation pannel)

- Si todos los bucles son vectorizados apropiadamente (>90% eficiencia): **rendimiento satisfactorio**
- Si no... En *Performance Issues* y panel *Recommendations* aparecen una serie de recomendaciones:
 - ¿Por qué no se vectoriza? Posibles dependencias, llamada a función, o que simplemente el compilador lo determina como ineficiente. Más información en URL
 - Si **vectoriza pero de forma ineficiente**: patrón de acceso a memoria ineficiente, tipo de conversiones de datos, falta de acceso a memoria alineado...

1.1.3.3 Modelo roofline

- Permite obtener modelo roofline
- Detectar posibles problemas (*Code Analysis*)

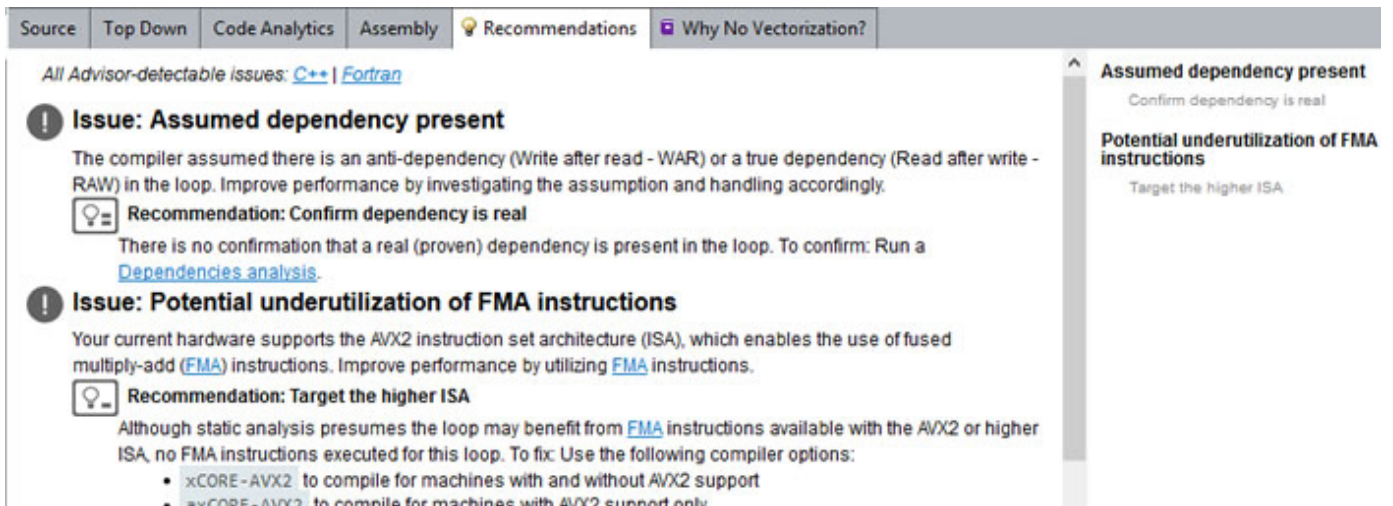


Figure 4: Intel Advisor-Panel Recomendación

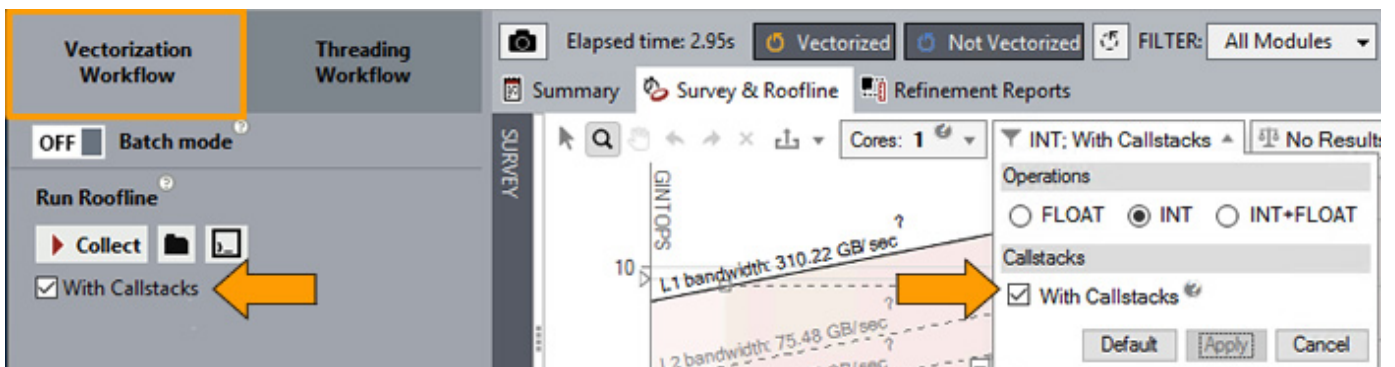


Figure 5: Intel Advisor-Roofline

1.1.3.4 Modelo de acceso a memoria

- Permite hacer un análisis de patrones de acceso a memoria: *Memory Access Patterns (MAP) Report*
 - *Uniform*: stride 0
 - *Unit stride* (stride 1)
 - *Constant stride* (stride N)
 - *Irregular stride*
 - *Gather (irregular) stride*

Site Location	Loop-Carried Dependencies	Strides Distribution	Access Pattern	Footprint Estimate			Site Name
				Max. Per-Instruction Addr. Range	First Instance Site Footprint	Simulated Memory Footprint	
[i] [o] [loop in matvec at Multiply.c...	No Information Available	100% / 0% / 0%	All Unit Strides	20B	40B	0B	loop_site...
[i] [o] [loop in matvec at Multiply.c...	No Information Available	100% / 0% / 0%	All Unit Strides	128B	256B	0B	loop_site...
[i] [o] [loop in matvec at Multiply.c...	No Information Available	No Strides Found	No Strides Found	0B	0B	0B	loop_site...
[i] [o] [loop in matvec at Multiply.c...	No Information Available	50% / 50% / 0%	Mixed Strides	4KB	6KB	0B	loop_site...

```

67 #pragma unroll
68 #pragma vector always
69 for (l = 0; l < size2; l++) {
70     b[l] += a[l][l] * x[l];
71 }

```

ID	Stride	Type	Source	Nested Function	Variable references	Max. Per-Instruction Addr. Range	Modules	Site Name	Access Type
[i] P2	376	Constant stride	Multiply.c:70		a	4KB	vec_samples.exe	loop_site_22	Read
[i] P7		Parallel site information	Multiply.c:69				vec_samples.exe	loop_site_22	
[i] P21	1	Unit stride	Multiply.c:70		x	128B	vec_samples.exe	loop_site_22	Read

Figure 6: Intel Advisor-Patrón de acceso a memoria

1.2 Ejercicio 4: Vectorización guiada

Existen ocasiones donde la vectorización automática no puede realizarse, bien por la complejidad del código en el que el compilador no es capaz de hacer uso del paralelismo SIMD, u otros factores que **inhiben la auto-vectorización** como uso de punteros, patrones de acceso irregulares, dependencias de datos u rendimiento esperado pobre. En estos casos el programador puede guiar al compilador mediante el uso de directivas o pragmas como `#pragma ivdep` o `pragma omp simd`. El código `guided_vec.cpp` presenta este problema:

```

user@lab:~ $ more guided_vec.optrpt
Begin optimization report for: compute_with_pointers(int, double*, double*, double*, double&)

LOOP BEGIN at guided_vec.cpp (9, 5)
    remark #25439: Loop unrolled with remainder by 4
LOOP END

LOOP BEGIN at guided_vec.cpp (9, 5)
<Remainder loop>
LOOP END

LOOP BEGIN at guided_vec.cpp (14, 5)
<Multiversiomed v2>
LOOP END

LOOP BEGIN at guided_vec.cpp (14, 5)
<Multiversiomed v1>
    remark #25228: Loop multiversiomed for Data Dependence
    remark #25564: Store sinked out of the loop
    remark #25439: Loop unrolled with remainder by 8

```

1.3 Ejercicio 5: Entrega k-nearest neighbors (KNN)

El algoritmo **K-Nearest Neighbors (KNN)** es un método de clasificación supervisado que asigna una etiqueta a una nueva instancia en función de la mayoría de sus vecinos más cercanos. Para ello vamos a describir el funcionamiento del algoritmo KNN:

1. **Preparación de Datos:**

- Se tiene un conjunto de datos con características (*features*) y etiquetas (*labels*). - Cada instancia del conjunto de datos es representada como un punto en un espacio multidimensional.

2. **Cálculo de Distancias:**

- Para clasificar un nuevo punto, se calcula la distancia entre este y todos los puntos del conjunto de entrenamiento.
- Se suele utilizar la **distancia Euclidia**, aunque también pueden emplearse otras métricas como la distancia Manhattan o Coseno.

3. **Selección de Vecinos Más Cercanos:**

- Se eligen los **K** puntos más cercanos en el conjunto de entrenamiento (donde **K** es un número entero definido por el usuario).

4. **Votación de Clases:**

- Se cuentan las etiquetas de los **K** vecinos más cercanos.
- La nueva instancia se clasifica en la categoría que tenga la mayoría de votos.

5. **Clasificación Final:**

- La instancia es asignada a la clase predominante entre los K vecinos.

Veamos con el ejemplo de la figura, como para clasificar un número punto (*punto azul*), buscamos los puntos con la distancia más cercada (k=5 significa los 5 vecinos más cercanos), y los clasificamos al nuevo punto con la clase predominante, en este ejemplo la clase verde.

1.3.1 Tarea

El entregable consistirá en modificar el código para que el alumno logre una implementación que explote de manera eficiente el procesamiento vectorial o SIMD, o bien de forma automática o guiada por medio de pragmas. Recomendaciones: - Evaluar las rutinas más costosas entre la medidas de tiempo, pudiendo hacer uso de los reportes o el profiler Intel-Advisor

```
clock_t start = clock();

// Classify each test sample
int k = 4;
for (int i = 0; i < test_data.n_samples; i++) {
    y_pred[i] = classify(train_data, test_data.samples[i].X, k, distances, labels);
}

clock_t end = clock();
double duration = (double)(end - start) / CLOCKS_PER_SEC;
```

- Posibles ineficiencias del código en la función `classify`:
 - Uso de memoria en Array-of-Structures que inhiban o reduzcan la aceleración potencial
 - Empleo de conversiones de datos
 - No utilización del repertorio de instrucciones del procesador
 - Dependencias de datos en bucles paralelos... ect

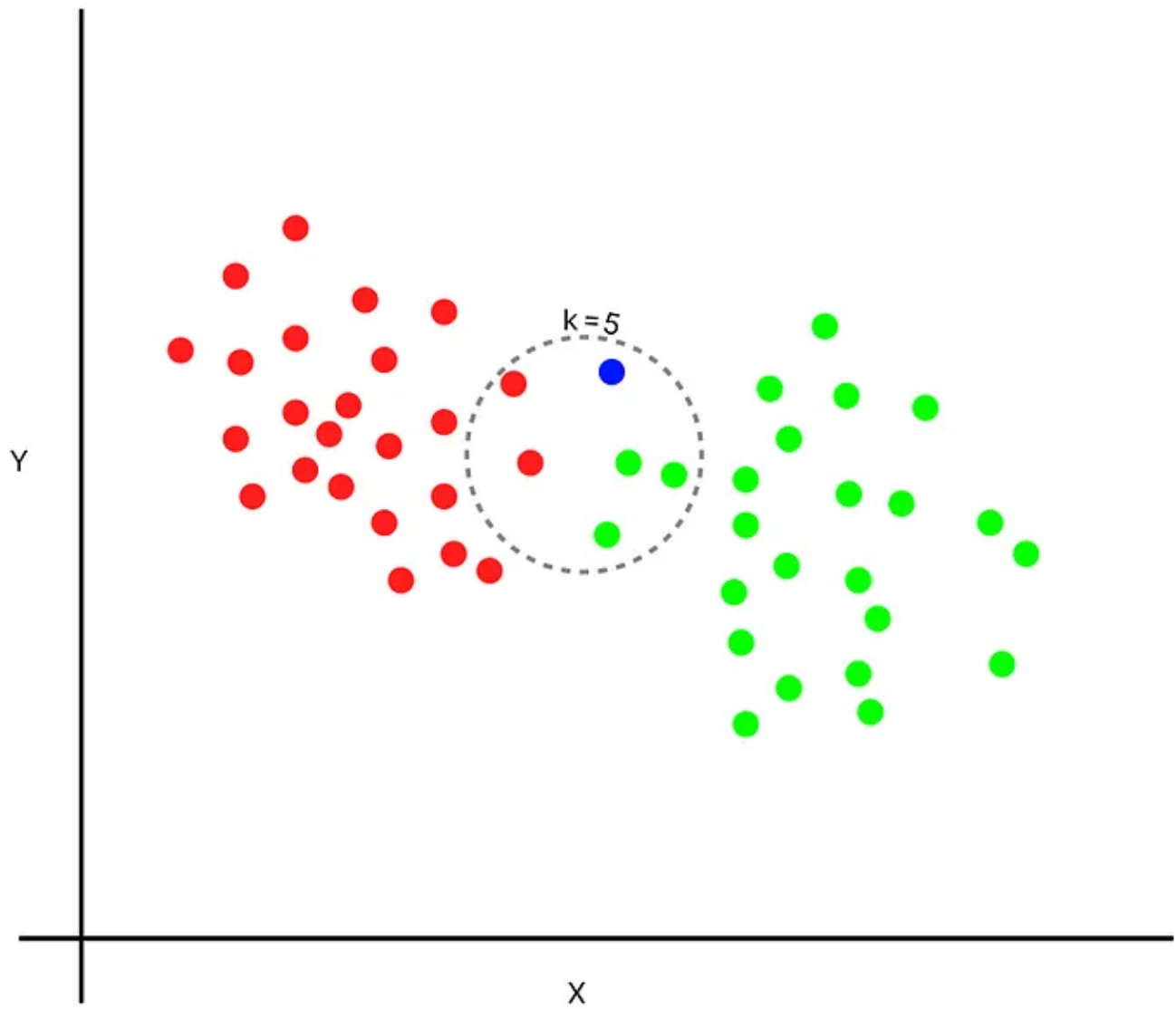


Figure 7: Ejemplo algoritmo KNN